

Taller Práctico de Repaso y Consolidación en C++: Recursividad, Búsqueda Secuencial y Punteros

Lucas Del Pino y Yamil De Jesus Ganduglia
Laboratorio de Programación (LPR) — 5° 3° A-B
E.E.S.T. N° 20 «Bangtan Sonyeondan», Vicente López
Prof. Yamil De Jesus Ganduglia
30 de septiembre de 2026

Introducción

El presente informe documenta la resolución de la Actividad 6 de Laboratorio de Programación (LPR), un taller de repaso en C++ que consolida tres conceptos de bajo nivel: la recursividad y el uso de la pila de llamadas, la búsqueda secuencial sobre arreglos de memoria contigua y el intercambio de valores mediante punteros. Para el trabajo se aplicó el Método 5R (registrar, reducir, responder, reflexionar y revisar): se compiló y ejecutó el código base, se identificó el núcleo de cada reto, se respondió el cuestionario técnico de control, se analizó su aplicación al Proyecto Integrador Anual (P.I.A.) y se documentó el resultado en un repositorio con control de versiones.

Desarrollo Repositorio y configuración Dual-Remote

El repositorio `repasogeneral` respeta la estructura exigida (`.gitignore`, `LICENSE`, `README.md`, `docs/`, `src/` y `capturas/`). Para protegerlo ante una caída de GitHub se configuró el modelo Dual-Remote: el remoto `origin` conserva una única URL de lectura (`fetch`) y dos URL de escritura (`push`), una de GitHub como servidor principal y otra de GitLab como respaldo, de modo que un solo `git push` actualiza ambos ([Git, s. f.](https://git-scm.com/docs/git-remote)). La Figura 1 muestra la verificación con `git remote -v`.

Figura 1

Verificación del Dual-Remote con `git remote -v`

```
PS C:\Users\Agustin\Desktop\Nueva carpeta (8)\repasogeneral> git remote -v
origin https://github.com/Pin0133/repasogeneral.git (fetch)
origin https://github.com/Pin0133/repasogeneral.git (push)
origin https://gitlab.com/Pin0133/repasogeneral.git (push)
PS C:\Users\Agustin\Desktop\Nueva carpeta (8)\repasogeneral>
```

Código y ejecución

El código fuente se encuentra en `src/main.cpp` y reúne los tres retos en una sola suite. Se compiló con `g++` sin errores ni advertencias (opciones `-Wall` y `-Wextra`). Al código base se le agregó una modificación propia en el Reto 2: la impresión de las direcciones de los primeros cuatro elementos del arreglo y de la distancia en bytes entre dos celdas consecutivas, con el fin de comprobar de forma directa que el arreglo ocupa memoria contigua (Pregunta 3). El Listado 1 muestra las dos funciones del taller y el Listado 2 la salida obtenida.

Figura 2

Ejecución de la Suite de Retos C++ en la Terminal de Comandos

```
int sumaRecursiva(int n) {
    if (n <= 0) return 0;           // Caso base
    return n + sumaRecursiva(n - 1); // Llamada recursiva
}

void intercambiarValores(int* ptrA, int* ptrB) {
    int temporal = *ptrA;
    *ptrA = *ptrB;
    *ptrB = temporal;
}

--- Verificacion de celdas contiguas
---&vectorDatos[0] = 0x7ffd5db35200
&vectorDatos[1] = 0x7ffd5db35204
&vectorDatos[2] = 0x7ffd5db35208
&vectorDatos[3] = 0x7ffd5db3520c
Distancia entre [0] y [1]: 4 bytes (sizeof(int) = 4)

=== RETO 3: INTERCAMBIO DE CELDAS ===
Valores previos -> X: 100 | Y: 500
```

Nota. Salida obtenida al compilar con g++. Las direcciones hexadecimales cambian en cada ejecución y entre sistemas operativos; lo que se mantiene constante es la diferencia de 4 bytes entre celdas contiguas.

Cuestionario Técnico de Control

Pregunta 1. Caso base: ¿qué error crítico ocurre en la RAM si una función recursiva carece de caso base?

Cada vez que una función se invoca, el sistema reserva en la pila (stack) un marco con sus parámetros, sus variables locales y la dirección de retorno. Ese marco solo se libera cuando la función termina. En una función recursiva sin caso base ninguna llamada llega a terminar: cada una invoca a otra nueva antes de retornar, por lo que los marcos se acumulan sin detenerse.

Como la pila es un sector de RAM de tamaño limitado, tarde o temprano se agota y se produce un desbordamiento de pila (stack overflow). El sistema operativo detecta el acceso fuera del espacio reservado y finaliza el programa de forma abrupta, por ejemplo con una falla de segmentación en Linux o con el código 0xC00000FD en Windows. En sumaRecursiva, si se eliminara la línea `if (n <= 0) return 0;`, el valor de `n` seguiría bajando hacia números negativos y las llamadas no se detendrían nunca. La Tabla 1 muestra la traza correcta, con caso base, para `n = 5`: la pila crece hasta seis marcos y luego se desarma devolviendo los valores acumulados.

Tabla 1*Traza de la pila de llamadas en sumaRecursiva(5)*

Llamada	Marcos en la pila	Valor Devuelto
sumaRecursiva(5)	1	$5 + 10 = 15$
sumaRecursiva(4)	2	$4 + 6 = 10$
sumaRecursiva(3)	3	$3 + 3 = 6$
sumaRecursiva(2)	4	$2 + 1 = 3$
sumaRecursiva(1)	5	$1 + 0 = 1$
sumaRecursiva(0) (caso base)	6	0

Nota. Los marcos se apilan de arriba hacia abajo y se desapilan en orden inverso, empezando por el caso base.

Pregunta 2. Eficiencia: ¿por qué break optimiza el rendimiento del microprocesador en una búsqueda?

La búsqueda secuencial compara el valor buscado con cada celda, en orden, hasta encontrarlo o agotar el arreglo; en el peor caso realiza n comparaciones, es decir, tiene complejidad $O(n)$ ([GeeksforGeeks, s. f.](https://www.geeksforgeeks.org/linear-search/)). La instrucción `break` interrumpe el ciclo en cuanto hay una coincidencia. Sin `break`, el ciclo seguiría recorriendo las celdas restantes aunque el resultado ya fuera conocido, y cada iteración extra implica trabajo para el microprocesador: lectura de memoria, comparación, incremento del índice y evaluación de la condición del ciclo.

En el programa, el valor 18 está en el índice 6: con `break` se realizan 7 comparaciones y sin él serían 10, de modo que se evitan 3 iteraciones inútiles (el 30 % del recorrido). En promedio, si el dato puede estar en cualquier posición, se recorre aproximadamente la mitad del arreglo. Conviene aclarar que `break` no cambia la complejidad asintótica en el peor caso (cuando el dato es el último o no existe), pero sí reduce el tiempo real de ejecución y el consumo de energía, algo relevante en sistemas embebidos.

Pregunta 3. Direccionamiento: ¿cómo comprobar con & que los elementos de un arreglo ocupan celdas contiguas?

Los elementos de un arreglo se almacenan uno a continuación del otro ([Microsoft, s. f.](https://learn.microsoft.com/en-us/cpp/cpp/arrays-cpp)). El operador `&` devuelve la dirección de memoria de una variable o de un elemento, de modo que al imprimir `&vectorDatos[i]` para índices consecutivos se comprueba que cada dirección supera a la anterior exactamente en `sizeof(int)`, que en este sistema son 4 bytes. La Tabla 2 lo muestra: terminan en 200, 204, 208 y 212. La relación general es dirección del elemento $i = \text{dirección base} + i \times \text{sizeof(int)}$. Por ejemplo, el valor 18 está en el índice 6, y $0x\dots200 + 6 \times 4 = 0x\dots200 + 0x18 = 0x\dots218$, que coincide con la dirección impresa por el Reto 2.

Otra forma de verificarlo es restar las direcciones convertidas a `char*`, como hace la modificación propia del código: la resta entre `&vectorDatos[1]` y `&vectorDatos[0]` da 4 bytes. Sin la conversión, la aritmética de punteros devuelve la distancia en elementos (1) y no en bytes ([cppreference.com, s. f.](https://en.cppreference.com/w/cpp/language/pointer)).

Tabla 2

Direcciones de los primeros elementos de vectorDatos

Índice	Valor	Dirección (hexadecimal)	Diferencia con la anterior
0	12	0x7ffd5db35200	—
1	45	0x7ffd5db35204	4 bytes
2	78	0x7ffd5db35208	4 bytes
3	90	0x7ffd5db3520c	4 bytes

Nota. Valores obtenidos en una ejecución con g++ en Linux. En otro sistema las direcciones serán distintas, pero la diferencia seguirá siendo de 4 bytes.

Pregunta 4. Variables temporales: ¿qué función cumple la variable temporal en el intercambio con punteros?

Al intercambiar dos valores es necesario conservar uno de ellos antes de sobrescribirlo. Si se escribiera directamente `*ptrA = *ptrB;`, el valor original de X (100) se perdería y la asignación siguiente, `*ptrB = *ptrA;`, copiaría el valor ya modificado: ambas variables terminarían valiendo 500. La variable temporal guarda una copia del valor de X antes de pisarlo y permite completar el intercambio en tres pasos, como muestra la Tabla 3.

La variable temporal es local de intercambiarValores, por lo que vive en el marco de pila de esa función y se destruye al retornar. El intercambio, en cambio, perdura porque las escrituras `*ptrA` y `*ptrB` se hacen sobre las direcciones de x e y, que pertenecen al marco de main. Si la función recibiera los parámetros por valor, modificaría solo copias y x e y quedarían sin cambios ([cppreference.com, s. f.](https://en.cppreference.com/w/cpp/language/pointer)).

Tabla 3*Traza del intercambio de valores con punteros*

Paso	Instrucción	Valor en x	Valor en y	Variable temporal
0	Estado inicial	100	500	—
1	<code>int temporal = *ptrA;</code>	100	500	100
2	<code>*ptrA = *ptrB;</code>	500	500	100
3	<code>*ptrB = temporal;</code>	500	100	100

Nota. x e y son las variables originales de main; temporal existe solo dentro de intercambiarValores.

Pregunta 5. Autoevaluación: ¿cuál de los tres retos resultó más complejo de comprender a bajo nivel y cómo se resolvió?

El reto que resultó más complejo de comprender a bajo nivel fue el **Reto 1: Recursividad**. La dificultad principal radicó en visibilizar el comportamiento interno de la pila de memoria (*Stack*), comprendiendo cómo cada llamada autorreferencial suspende la ejecución actual y reserva un nuevo marco de pila (*Stack Frame*) con sus propias variables locales hasta alcanzar el caso base.

Para resolver este obstáculo, aplicamos la metodología del taller: ejecutamos el código en la terminal de VS Code inspeccionando el flujo de retorno, graficamos a mano el diagrama de bloques de la pila de memoria (traza de memoria) y consultamos la documentación técnica sobre el ciclo de vida de las funciones en RAM. Como resultado, logramos comprender que el caso base no es solo un if de parada, sino la condición indispensable que inicia el desapilado de marcos y evita el colapso de memoria por Stack Overflow.

Figura 3

Traza de memoria RAM dibujada a mano

Conexión con el Proyecto Integrador Anual

o, Los tres conceptos tienen aplicación directa en un prototipo con sensores o módulos de hardware. La búsqueda secuencial sirve para recorrer un búfer de lecturas (por ejemplo las últimas N mediciones de un sensor) y detenerse con break al detectar el primer valor que supera un umbral. Los punteros permiten que una función reciba la dirección de una variable y la actualice directamente, sin copiarla, o que intercambie posiciones dentro de un búfer. La recursividad, en cambio, debe usarse con cautela en microcontroladores, donde la pila es muy pequeña (un Arduino Uno, por ejemplo, dispone de solo 2 KB de SRAM compartidos entre pila y variables), y en esos casos suele preferirse una solución iterativa.

Aclaración sobre el P.I.A.: Por el momento no se utiliza hardware en el proyecto integrador.

Conclusiones

El taller permitió relacionar cada concepto con lo que ocurre en la memoria. La recursividad necesita un caso base porque la pila es finita; la búsqueda secuencial se beneficia de interrumpir el ciclo cuando ya halló el dato, y los arreglos pueden recorrerse con aritmética de direcciones porque ocupan celdas contiguas; los punteros, junto con la variable temporal, permiten modificar variables originales fuera de su ámbito local. Además, el uso de Git con Dual-Remote y de un CHANGELOG.md protege y documenta el trabajo de cara al Proyecto Integrador Anual.

Referencias

cppreference.com. (s. f.). *Pointer declaration*.

<https://en.cppreference.com/w/cpp/language/pointer>

GeeksforGeeks. (s. f.). Linear search algorithm. <https://www.geeksforgeeks.org/linear-search/>

Git. (s. f.). git-remote Documentation. <https://git-scm.com/docs/git-remote>

Microsoft. (s. f.). Arrays (C++). Microsoft Learn.

<https://learn.microsoft.com/en-us/cpp/cpp/arrays-cpp>